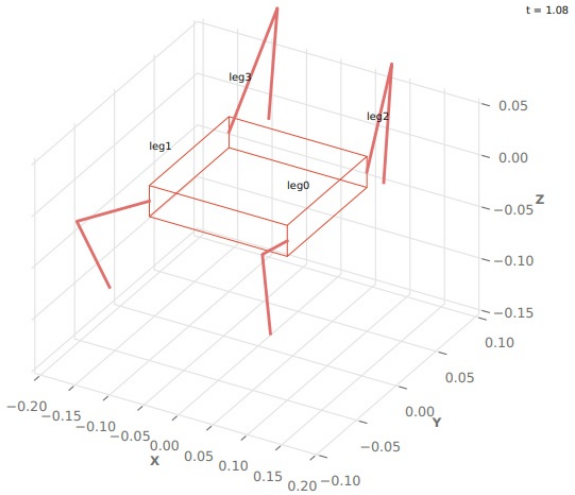
TEL200 - Introduction to Robotics

Group 5



Walking Robot Project

Authors

Andreas Brustad
Johannes Standal
Håkon Bekken

Norwegian University of Life Sciences

1 Abstract

This report concerns the design and implementation of a simulated quadruped walking robot. The project covers three main topics: motion planning for coordinated leg movement, path planning for navigation, and localization and mapping based on real lidar data. Together, these form the core components of an autonomous robotic system.
Project links: Bexter, 2026, RealHaakon, 2026

2 Introduction

2.1 Motion Planning

A quadruped walking robot requires coordinated control of four legs, each kinematically equivalent to a three-joint serial-link manipulator. Gait generation ensures static stability by keeping at least three feet in contact with the ground at all times, dividing each leg's motion into a stance phase and a reset phase. The resulting crawl gait is achieved by assigning each leg a phase-shifted version of the same cyclic foot trajectory.

2.2 Path Planning

For a robot to navigate an environment, it must plan a collision-free path from a start position to a goal. The Probabilistic Roadmap (PRM) method addresses this by constructing a sparse graph of obstacle-free paths through the free space during an offline planning phase. Once built, the roadmap can be reused to answer any start-to-goal query efficiently, making PRM well suited to large-scale and high-dimensional planning problems.

2.3 Localization and Mapping

Autonomous navigation requires a robot to know both its own position (localization) and the structure of its environment (mapping). These two problems are interdependent: a map is needed for localization, and a pose estimate is needed for mapping. For the particular problem using the real lidar data over Killian court, an evaluation between PRM and A* search algorithm is made

3 Method

3.1 Motion Planning

A robot leg is kinematically equivalent to a robot arm. For a statically stable quadruped, a three-joint serial-link manipulator is sufficient, since the foot makes point contact with the ground and orientation is not critical. (Corke, 2023)

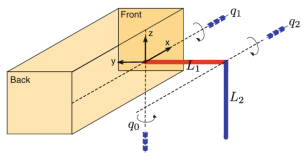


Figure 1: Illustration of frame and axis rotations for the simple leg (Corke, 2023)

The coordinate frame is defined with the x-axis pointing forward, the z-axis upward, and the y-axis to the left. The leg link lengths are $L_1 = L_2 = -100\,\text{mm}$. The hip has two degrees of freedom: rotation about the z-axis (forward/backward) and rotation about the x-axis (up/down). The knee is offset by L_1 in the y-direction and rotates about the x-axis, while the foot is offset by L_2 in the z-direction. In ETS notation:

$$\texttt{leg} = R_z \cdot R_x \cdot T_y(L_1) \cdot R_x \cdot T_z(L_2) \tag{1}$$

At zero joint angles, forward kinematics yields a foot position of $(0,\ -0.1,\ -0.1)\,\text{m}$, confirming the design.

During the *stance phase*, the foot remains in contact with the ground 50 mm below the hip, moving from 50 mm forward to 50 mm behind the hip to propel the body forward. During the *reset phase*, the foot lifts to 20 mm below the hip, swings forward, and is placed back on the ground. To maintain static stability, at least three feet must remain on the ground at all times, so each leg spends 75% of the cycle in stance and 25% resetting. This coordinated pattern is known as a *crawl gait* (or *wave gait*).

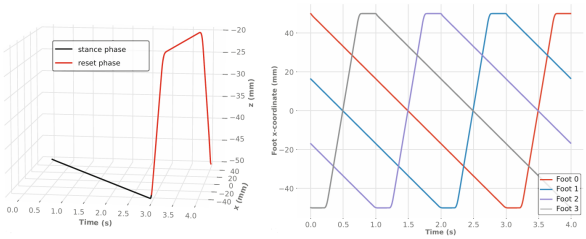


Figure 2: Trajectory of a single foot and motion of each leg (Corke, 2023)

The foot path is defined by five via points forming a closed cycle:

$$\mathbf{p}_{\text{via}} = \begin{bmatrix} x_f & y & z_d \\ x_b & y & z_d \\ x_b & y & z_u \\ x_f & y & z_u \\ x_f & y & z_d \end{bmatrix} \tag{2}$$

where $x_f = 50\,\text{mm}$, $x_b = -50\,\text{mm}$, $y = -50\,\text{mm}$, $z_u = -20\,\text{mm}$, and $z_d = -50\,\text{mm}$. The path is sampled at 100 Hz using `mstraj` with segment times $[3,\ 0.25,\ 0.5,\ 0.25]\,\text{s}$, giving a 4 s cycle of 400 points. Joint angle trajectories are obtained by numerical inverse kinematics, solving only for end-effector translation:

$$\texttt{mask} = [1,\ 1,\ 1,\ 0,\ 0,\ 0] \tag{3}$$

The robot body has width $W = 100\,\text{mm}$ and length $L = 200\,\text{mm}$. Four leg instances are placed at the body corners using base transforms. Legs on the left side (legs 2 and 3) are rotated by π about the z-axis so they point outward from the body. Each leg runs the same cyclic trajectory with a phase offset of one quarter of the cycle (100 points), achieved via modulo indexing:

$$k' = (k + \phi) \mod N \tag{4}$$

where k is the current time step, $\phi \in \{0, 100, 200, 300\}$ is the phase offset per leg, and $N = 400$ is the total cycle length. For legs on the right-hand side, the sign of the first joint angle is flipped to mirror the motion symmetrically.

For global positioning the robot is modelled as *unicycle*, a common kinematic abstraction for mobile robots moving on a planar surface. The robot pose is fully described by three states:

$$\mathbf{x}_v = (x,\ y,\ \theta)^T \tag{5}$$

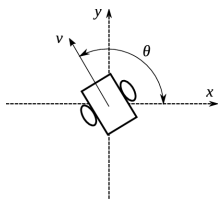


Figure 3: Illustration of pose using unicycle model (Li, 2017)

where x and y are the position coordinates in the global reference frame and θ is the heading angle relative to the x-axis.

3.2 Motion Primitives

The global motion of the robot is defined by two discrete motion primitives for forward translation of 10 cm, and in-place rotation of 1°. Each primitive corresponds to one full gait cycle. A forward cycle produces a 10 cm displacement in the current heading direction, while a rotation cycle changes the orientation by 1°. The robot pose is updated using the unicycle model.

$$x_{k+1} = x_k + \Delta s \cos\theta_k, \qquad y_{k+1} = y_k + \Delta s \sin\theta_k, \qquad \theta_{k+1} = \theta_k + \Delta\theta \tag{6}$$

Utilizing these motion primitives, we can achieve planar motion by combining forward and rotational steps. However since both translation and rotation are reduced to certain numbers, some loss of precision occurs.

Forward motion is performed in steps of 100 mm. This means the robot cannot move an exact distance smaller than 100 mm. When approaching a waypoint, the maximum

positional overshoot is therefore limited to approximately 50 mm (half a step). Similarly, rotational motion is limited to integer degrees, making it impossible to represent non-integer heading angles exactly.

Over longer paths, these small discretization effects may accumulate and cause deviation from the ideal continuous trajectory. To reduce this effect, the robot continuously tracks its position relative to the next node in the PRM path. Any small translational or angular overshoot is naturally corrected when aligning toward the following node.

Although this approach reduces mobility resolution, the resulting error remains bounded and acceptable for the scale of the simulated environment.

3.3 Path Planning

Robot path planning is divided into two phases: a *planning phase*, which analyses the map and constructs an obstacle-free network, and a *query phase*, which finds a path between a given start and goal using that network. Roadmap methods decouple these phases, so the roadmap is computed once and reused for any start and goal combination. (Corke, 2023)

The Probabilistic Roadmap (PRM) method builds the roadmap by randomly sampling free-space vertices and connecting them with obstacle-free straight-line edges, forming an embedded graph. With sufficiently many vertices, the graph spans the entire free space. The connectivity depends on the number of sampled points; too few vertices may result in disjoint components, preventing valid paths from being found.

Once the roadmap is constructed, the query phase finds a path by connecting the start position to the nearest roadmap vertex, traversing the graph via a minimum-distance route, and exiting at the vertex closest to the goal. The path is represented as an ordered list of via points:

$$\mathbf{p} = \{\mathbf{p}_0,\ \mathbf{p}_1,\ \ldots,\ \mathbf{p}_n\} \tag{7}$$

Let the robot pose be (x_r, y_r, θ_r) and the next waypoint (x_t, y_t). The required walking and turning cycles are computed as

$$d = \sqrt{(x_t - x_r)^2 + (y_t - y_r)^2}, \qquad N_{\text{walk}} = \text{round}\left(\frac{d}{100\,\text{mm}}\right), \tag{8}$$

$$\Delta\theta = ((\text{atan2}(y_t - y_r, x_t - x_r) - \theta_r + \pi) \bmod 2\pi) - \pi, \qquad N_{\text{turn}} = \text{round}\left(\frac{\Delta\theta}{1^\circ}\right). \tag{9}$$

3.4 Localization and Mapping

Robot navigation requires two fundamental capabilities: knowing where the robot is (localization) and knowing what the environment looks like (mapping). These problems are deeply interdependent — a map is needed for accurate localization, while a reliable pose estimate is needed to build a map. (Corke, 2023)

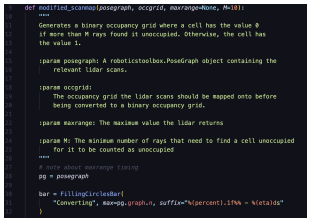
With the lidar data provided by the robot, we can map the area onto an occupancy grid in python using the scanmap() method. The occupancy grid is represented as a grid of cells, where each cell has a value indicating its occupancy. The way this value is calculated is

by first going through the rays and using the Bresenham line-drawing algorithm to find the cells each ray passes through. The cell at the end of its path has its value reduced by one, and the rest of the cells on the path has their respective values increased by one. Through this incrementation, each cell that has rays pass through them ends up with a value indicating how occupied it is. In the scanmap() method, it then goes on to clamp the values to be either -1, 0 or 1. If the cell's value is negative, it is assumed to be occupied, and its final value is -1. If the cell's value is zero, it is decided to be unknown, and the value stays at zero. And finally, if the cell's value is positive, it is assumed to be free space, and the value is set to 1.

To modify the scanmap() method to arrive at a binary occupancy grid, where if more than M lidar scans confirm a cell is free, the cell has value 0, otherwise 1, a few steps were taken:

- The decrementation of cells needs to be removed, as the task description states that a cell is free if more than M rays pass through it, not if at least M more rays pass through than don't.
- In the original implementation it only goes through 20% of the rays. This needs to be modified to go through all rays.
- The final step in the scanmap() method needs to be modified for a threshold of M rather than 0. The clamp in the final step also needs to be modified to clamp to 0 and 1 to make it binary. And the final step also needs to be modified to return a binary occupancy grid.

The implementation of these modifications is given as follows:



```
def modified_scanmap(posegraph, occgrid, maxrange=None, M=10):
    """
    Generates a binary occupancy grid where a cell has the value 0
    if more than M rays found it unoccupied. Otherwise, the cell has
    the value 1.

    :param posegraph: A roboticstoolbox.PoseGraph object containing the
        relevant lidar scans.

    :param occgrid:
        The occupancy grid the lidar scans should be mapped onto before
        being converted to a binary occupancy grid.

    :param maxrange: The maximum value the lidar returns

    :param M: The minimum number of rays that need to find a cell unoccupied
        for it to be counted as unoccupied
    """
    # note about maxrange timing
    pg = posegraph

    bar = FillingCirclesBar(
        "Converting", max=pg.graph.n, suffix="%(percent).1f%% - %(eta)ds"
    )
```

(a) Modified scanmap() part 1

```
    grid1d = occgrid.ravel
    for i in range(pg.graph.n):

        xy = pg.scanxy(i)
        r, theta = pg.scan[i]
        if maxrange is not None:
            toofar = np.where(r > maxrange)[0]
            xy = np.delete(xy, toofar, axis=1)
        xyt = pg.vindex[i].coord

        xy = SE2(xyt) * xy

        # start of each ray
        p1 = occgrid.w2g(xyt[:2])

        for p2 in occgrid.w2g(xy.T):

            # all cells along the ray
            try:
                k = occgrid._line(p1, p2)
            except ValueError:
                # line() ir ignore rays to points outside the grid map
                continue

            # increment cells along the ray, these are free space
            grid1d[k[1:-1]] += 1

        bar.next()
    bar.finish()
```

(b) Modified scanmap() part 2

```
    grid1d[grid1d <= M] = 1
    grid1d[grid1d > M] = 0
    bar.finish()
    return rtb.BinaryOccupancyGrid(
        grid=occgrid.grid,
        cellsize=0.1
    )
```

(c) Modified scanmap() part 3

To implement a PRM path-planner for the free space in this occupancy grid, we create a PRMPlanner object linked to the binary occupancy grid from modified_scanmap. Because there is a lot of free space outside the intended area, npoints is set rather high in order to make proper coverage of the intended area more probable.

An alternative path planning algorithm that can be more efficient than PRM is A^*. The way this works is you store three essential values for each cell in the grid:

$$g(n) \rightarrow \text{The calculated cost from start to here}$$
$$h(n) \rightarrow \text{The estimated cost from here to goal}$$
$$f(n) \rightarrow \text{The sum of } g \text{ and } h\text{, the total estimated cost of path}$$

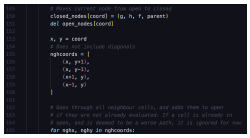
You sort these into two different categories: Open, which are yet to be evaluated, and closed, which have been evaluated. You evaluate the neighbour cells of the current cell in this way until you've either reached the goal, and thus have found the cheapest path, or until the open category is empty, in which case there is no path to the goal. The python implementation of this is as follows:

```
def astarplanning(occgrid: rtb.OccupancyGrid, start, goal):
    # Start format:
    # (theta(degrees), x(m), y(m), parent_coordinates)
    start = tuple(occgrid.w2g(start).tolist())  # Convert to grid coords
    goal = tuple(occgrid.w2g(goal).tolist())  # Convert to grid coords
    goal_x, goal_y = goal
    start_x, start_y = start
    g_start = 0 # The cost to move from start to start is zero
    # Heuristic: Manhattan distance
    h_start = abs(start_x - goal_x) + abs(start_y - goal_y)
    f_start = g_start + h_start  # Estimated total cost to target
    open_nodes = {(start): (g_start, h_start, f_start, None)}
    closed_nodes = {}

    # Log the start and goal for debugging
    pbar = tqdm(total=inf_start, desc="Planning, Distance to target: ")
```

Figure 5: A^* algorithm part 1

```
    while len(open_nodes) > 0:
        coord = None
        lowestf = float('inf')

        # Makes the current node the one with the lowest
        # heuristically estimated distance to target
        for tent_coord, values in open_nodes.items():
            g, h, f, parent = values
            if f < lowestf:
                lowestf = f
                coord = tent_coord

        g, h, f, parent = open_nodes[coord]

        # Reached goal, constructs path by recursively going
        # back to start through node parents
        if coord == goal:
            pbar.close()
            path = [occgrid.g2w(coord)]
            while parent is not None:
                path.append(occgrid.g2w(parent))
                g, h, f, parent = closed_nodes[parent]
            path.reverse()
            occgrid.plot(cmap="gray")
            x_coords, y_coords = zip(*path)
            plt.plot(x_coords, y_coords, linewidth=3)  You,
            plt.plot(path[0][0], path[0][1], "bo")  # Startpoint
            plt.plot(path[-1][0], path[-1][1], "go")  # Endpoint
            plt.show()
            return

        # Moves current node from open to closed
        closed_nodes[coord] = (g, h, f, parent)
        del open_nodes[coord]
```

Figure 6: A^* algorithm part 2



```
        # Moves current node from open to closed
        closed_nodes[coord] = (g, h, f, parent)
        del open_nodes[coord]

        x, y = coord
        # Does not include diagonals
        nghcoords = [
            (x, y+1),
            (x, y-1),
            (x+1, y),
            (x-1, y)
        ]

        # Goes through all neighbour cells, and adds them to open
        # if they are not already evaluated. If a cell is already in
        # open, and is deemed to be a worse path, it is ignored for now
        for nghx, nghy in nghcoords:
```

Figure 7: A^* algorithm part 3

```
        # Goes through all neighbour cells, and adds them to open
        # if they are not already evaluated. If a cell is already in
        # open, and is deemed to be a worse path, it is ignored for now
        for nghx, nghy in nghcoords:
            if (nghx, nghy) in closed_nodes:
                # Already evaluated
                continue
            if occgrid.grid[nghy, nghx]:
                # Cell is wall
                continue
            if not (0 <= nghx < occgrid.grid.shape[1] and
                0 <= nghy < occgrid.grid.shape[0]):
                # Out of bounds
                continue
            ngh_g = g + 1
            if (nghx, nghy) in open_nodes:
                tent_g, tent_h, tent_f, tent_parent = open_nodes[(nghx, nghy)]
                if ngh_g >= tent_g:
                    # Not a better path
                    continue
            # Calculate heuristic and f
            ngh_h = abs(nghx - goal_x) + abs(nghy - goal_y)
            ngh_f = ngh_g + ngh_h
            open_nodes[(nghx, nghy)] = (ngh_g, ngh_h, ngh_f, coord)
        pbar.update(1)
    # Never reached goal
    pbar.close()
    print("Path not found")
```

Figure 8: A^* algorithm part 4

4 Results

4.1 Motion Planning

The implemented crawl gait produced stable forward motion in the simulation. At all times, at least three legs remained in contact with the ground, ensuring static stability. No tipping or oscillatory instabilities were observed during straight-line walking.

Each gait cycle resulted in an approximate forward displacement of 100 mm, as intended by the motion primitive design. Small deviations from the ideal displacement were observed due to numerical inverse kinematics and floating-point rounding, but these errors remained negligible over short distances. The turning was also affected by floating point precision errors, however the angular error did not cause instability.

When combining rotation and translation primitives, the robot was able to approximate arbitrary planar trajectories. Although the motion is piecewise discrete rather than continuous, the resulting path closely resembled the desired trajectory for the scale of the simulated environment.

Overall, the motion primitive framework provided predictable and stable locomotion, with bounded discretization error that was acceptable for subsequent path-following tasks.

4.2 Path Planning

The PRM graph contains a parameter specifying the number of nodes in the roadmap. The objective is to determine a suitable node count for the given house map. If too few nodes are used, the graph may contain disconnected subgraphs (see Fig. 9), causing large parts of the environment to become unreachable. On the other hand, using too many nodes increases computational cost and reduces performance.

To determine a suitable value, one could use an analytical iterative approach. However, for this project an empirical approach was chosen, as it is sufficient for evaluating performance in a fixed environment.

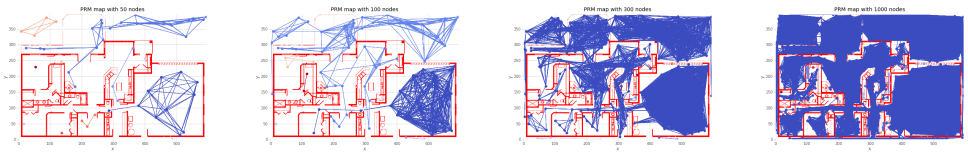


Figure 9: Low node counts result in disconnected components, while higher node counts improve connectivity at the cost of computation time.

From Fig. 9, it can be observed that 300 nodes provide sufficient connectivity throughout the house without producing disconnected subgraphs. This enables path planning between almost any two points in the environment while keeping computation time reasonable.

Using the PRM planner with 300 nodes, five different paths were generated for the robot to follow, as shown in Fig. 10.

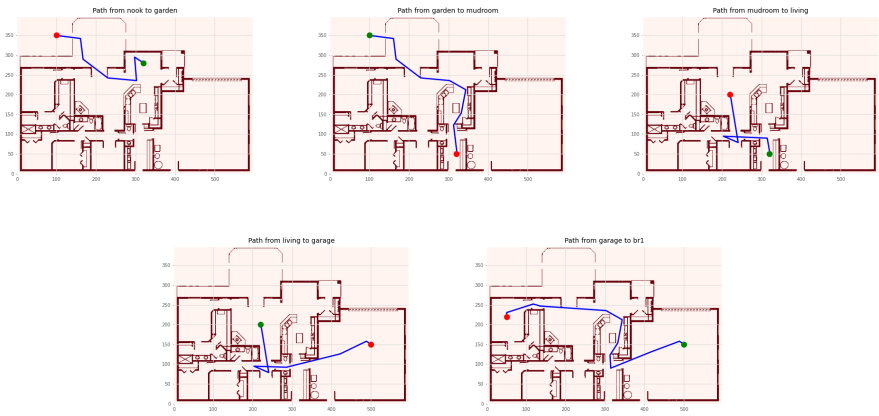


Figure 10: Five example paths generated using the PRM with 300 nodes.

To execute a planned path, the robot is first placed at the start coordinate using the custom `GOTO(x, y)` method in the `WalkingRobot` class. The robot continuously evaluates its own pose relative to the next waypoint in the path. This compensates for the accumulated discretization error introduced by the motion primitives described in Section 3.2.

The robot executes the primitives sequentially: it first performs the required number of turning cycles, then executes the walking cycles. This process is repeated for each waypoint until the final node in the path is reached.

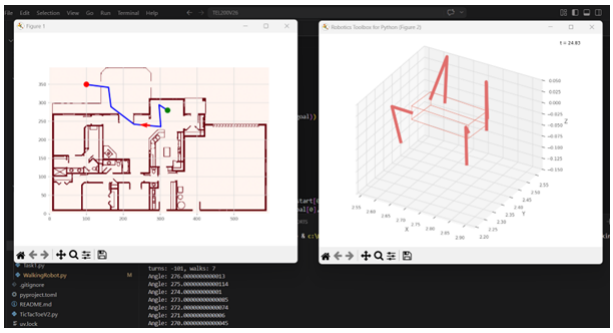
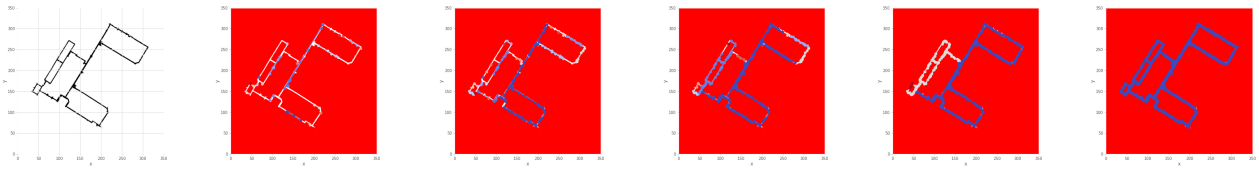


Figure 11: Walking robot following a path

4.3 Localization and mapping

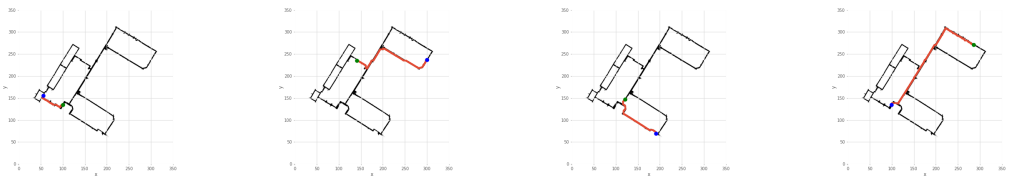
Generating PRM-planners with 50, 150, 300, 500 and 1000 nodes led to the following results, including the base map generated by modified_scanmap():



(a) Killian Map (b) PRM n=50 (c) PRM n=150 (d) PRM n=300 (e) PRM n=500 (f) PRM n=1000

Figure 12: Killian map binary occupancy grid and PRM graphs

With four different cases of randomly generated start coordinates (blue point), and randomly generated goal coordinates (green point), the results of the A^* algorithm are as follows given in red lines:



5 Discussion

5.1 Motion Planning and Path Following

The implemented leg kinematics produced stable motion in simulation. While the model behaves consistently in the simulated environment, it is likely that a physical implementation would require more refined modeling, better dynamics handling, and feedback control to achieve smooth and realistic motion.

A major limitation of the system is the use of only two motion primitives: 100 mm forward translation and 1° in-place rotation. Although this simplifies control, it significantly limits motion resolution. The robot cannot perform fine adjustments efficiently. Turning is slow because large rotations require many 1° cycles. A more flexible primitive set (e.g., 30°, 10°, 5°, and 1° rotations) would improve both speed and performance.

During path execution, the robot was generally able to follow the planned paths. However, small overshoot and undershoot effects were observed due to discretization of motion. These errors remained bounded and were partially corrected when the robot aligned toward the next waypoint.

The generated PRM paths were valid but not always smooth. Since the robot executes a turn-then-walk strategy at each waypoint, the overall motion becomes piecewise and slow. Additionally, the PRM struggles in narrow rooms and near corners, where random sampling may fail to provide sufficient coverage. As a result, some regions close to walls are

difficult or impossible to reach. However as a side bonus this can make up for the absence of collision detection, by making the robot less likely to walk near walls and corners.

In general the system is modular and well structured. The separation between gait generation, motion primitives, and path planning makes it straightforward to replace the PRM algorithm or extend the motion primitive set. Despite its simplifications, the robot successfully completed all required tasks in the assignment.

5.2 Localization and mapping

From Fig. 12, the binary occupancy grid produced by the modified scanmap() method captures the main structure of the Killian hallways. Walls are clearly defined, and the free space inside corridors is visible. This indicates that the lidar processing and thresholding approach works as intended for generating a usable map representation.

From the PRM map attempts it can also be observed that even with a huge amount of nodes, the PRM is still unable to provide sufficient connectivity throughout the Killian hallways. This drastically increases the chance of an error when attempting to plan a path between two points in the environment. This is because tight corridors and sharp turns require more points concentrated around them in order to establish connectivity. Therefore, the amount of points needs to be quite large for the probability of connection to be at a reliable level.

The A^* algorithm on the other hand is able to cover the entire grid, and with it's best first search approach is likely to find the best path quite quickly. Because of the tight hallways and sharp turns of the Killian court, the A^* algorithm is therefore more efficient to consider.

6 Bibliography

References

Li, H. (2017). Unicycle model of a wheeled mobile robot [Accessed: 2026-04-24]. https://www.researchgate.net/figure/Unicycle-model-of-a-wheeled-mobile-robot_fig54_319477982

Corke, P. (2023). Robotics, vision & control: Fundamental algorithms in python (3rd ed.). Springer Nature. https://doi.org/10.1007/978-3-031-06469-2

Bexter. (2026). Tel200 walking robot video. https://www.youtube.com/watch?v=s7YNcI4iuiI

RealHaakon, S., JohannesStandal. (2026). Tel200v26. https://github.com/RealHaakon/TEL200V26